

Section - 3 (4*5)

Coding Questions 5 Marks

Q1 .Given an integer array , find the largest positive integer k such that -k also exists in the array.Return the positive integer k.

If there is no such integer, return -1

Input Format

First line - Take size of array N.

Second line - Take N integer value to store them in an array.

Output Format

Print the required output if it is present and -1 if such number is not present.

Constraints

1. $1 \leq N \leq 10^3$
2. $1 \leq \text{arr}[i], X \leq 10^4$

Input	Output
7 -1 -3 -2 0 1 3 2	3

Solution

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    int m;
```

```
cin >> m;
```

```
int arr[m];
```

```
for (int i = 0; i < m; i++) {
```

```
    cin >> arr[i];
```

```
}
```

```
int largest = 0;
```

```
bool found = false;
```

```
for (int i = 0; i < m; i++) {
```

```
    for (int j = 0; j < m; j++) {
```

```
        if (arr[j] == -arr[i]) {
```

```
            found = true;
```

```
            if (arr[i] > largest) {
```

```
                largest = arr[i];
```

```
            }
```

```
            break;
```

```
        }
```

```
    }
```

```
}
```

```
if (found) {
```

```
    cout << largest << endl;
```

```
} else {
```

```
    cout << -1 << endl;
```

```
}
```

```
return 0;
```

```
}
```

Test Cases

Input	Output
6 5 1 2 3 -1 -2	2
10 10 5 10 -5 -10 15 -15 20 -20 25	20
10 9 7 12 -7 -12 15 -15 18 -18 21	18
8 1 2 3 4 5 6 7 8	-1
12 1 3 5 7 9 11 -1 -3 -5 -7 -9 -11	11
4 -1 -1 -1 -1	-1
6 100 -200 300 -100 200 -300	300

Ques 2 - Raj works in a MNC. His boss has told him to select some contiguous values from given n values such that the sum of all these values

is maximum. Note that all these values need to be contiguous i.e. consecutive. Find the maximum sum that Raj can obtain.

Input Format

First line - Take size of array N.

Second line - Take N integer value to store them in an array.

Output Format

Print the maximum sum possible from all the subarrays.

Constraints

$1 \leq \text{arr.size()} \leq 10^5$

$-10^3 \leq \text{arr}[i] \leq 10^3$

Input	Output
5 1 2 3 -2 5	9

Solution

```
#include<iostream>
```

```
using namespace std;
```

```
int maxSubarraySum(int arr[], int n){
```

```
    int maxending =0;
```

```
    int max_so_far = arr[0];
```

```
    int curr_max = arr[0];
```

```
    for(int i=1; i<n; i++){
```

```
        curr_max = max(arr[i], curr_max+arr[i]);
```

```

        max_so_far = max(max_so_far, curr_max);
    }

    return max_so_far;
}

int main(){
    int n;

    cin>>n;

    int arr[n];

    for(int i=0; i<n; i++){
        cin>>arr[i];
    }

    cout << maxSubarraySum(arr, n) << endl;

    return 0;
}

```

Test Cases

<u>Input</u>	<u>Output</u>
6 -1 -2 -3 -4 -5 -6	-1
12 2 1 -2 55 -13 21 18 10 -27 17 9 2	93

12 -10 -20 -30 -40 -50 -60 -70 -80 -90 - 100 -110 -120	-10
15 10 20 30 40 50 60 70 80 90 100 110 120 130 140 150	1200
19 15 -12 8 -17 19 20 -16 14 13 -11 10 9 -8 7 6 -5 4 3 -2 1	65
16 -100 200 -300 400 -500 600 -700 800 -900 1000 -1100 1200 -1300 1400 -1500 1600	1600

Ques 3 - Rotate matrix 90 anticlockwise :

Given a square matrix `mat[][]`, turn it by 90 degrees in an anticlockwise direction Note: it is a square matrix so , the number of rows must be equal to the number of columns.

Input Format

First line - Take size of array N.

Second line - Take N*N integer value to store them in a 2D array or square matrix.

Output Format

Print the rotated matrix in proper format.

First print 1st row in first line , second row in next line and so on. (refer sample output to print required output)

Constraints

$$1 \leq N \leq 7$$

$$0 \leq \text{arr}[i] \leq 10^3$$

Input	Output
9 7 1 5 2 9 4 11 8 0	5 4 0 1 9 8 7 2 11
9 17 11 9 3 3 3 3 2 1	9 3 1 11 3 2 17 3 3

Solution

```
// rotate the matrix anticlockwise.
```

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
void rotate90(vector<vector<int> >& mat) {
```

```
    int n = mat.size();
```

```
vector<vector<int> > res(n, vector<int>(n));
```

```
for (int i = 0; i < n; i++) {
```

```
    for (int j = 0; j < n; j++) {
```

```
        res[n - j - 1][i] = mat[i][j];
```

```
    }
```

```
}
```

```
mat = res;
```

```
}
```

```
int main(){
```

```
    int N;
```

```
    cin>>N;
```

```
    vector<vector<int> >ans(N, vector<int>(N));
```

```
    for(int i = 0; i < N; i++) {
```

```
        for(int j = 0; j < N; j++) {
```

```
            cin>>ans[i][j];
```

```
        }
```

```
    }
```

```
    rotate90(ans);
```

```
    for(int i = 0; i < N; i++) {
```

```
        for(int j = 0; j < N; j++) {
```

```
            cout<<ans[i][j]<<" ";
```

```
        }
```

```
    cout << endl;
```

```
}
```

```
return 0;
```


}

Test Cases

<u>Input</u>	<u>Output</u>
3 1 2 3 4 5 6 7 8 9	3 6 9 2 5 8 1 4 7
3 7 1 5 2 9 4 11 8 0	5 4 0 1 9 8 7 2 11
5 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25	5 10 15 20 25 4 9 14 19 24 3 8 13 18 23 2 7 12 17 22 1 6 11 16 21
4 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16	4 8 12 16 3 7 11 15 2 6 10 14 1 5 9 13
4 1 2 3 4 1 2 3 4 1 2 3 4 1 2 3 4	4 4 4 4 3 3 3 3

	2 2 2 2 1 1 1 1
3 9 9 9 9 9 9 9 9	9 9 9 9 9 9 9 9 9

Ques 4 - Check if we get hacked or not :

There was a data breach on your browser and all the user details about affected users is shared by the company but

It is not organized and you want to check if your name is in the compromised list or not.

Input Format

First line - You have to take a string that have name of all the user's usernames in random order separated by #.

Second line - you have to take the string which you want to check if hacked or not.

Output Format

Print “ Hacked “ if your data got hacked and “ Not Hacked “ if your data is not hacked.

Means your name is not in those names, if hacked then you need to return starting position

From where your name is matched

.

Constraints

$1 \leq \text{string length} \leq 10^5$

Input	Output
jitendra23#kunal@32#rajesh_56#kartik_007 jitendra23	Hacked 1

Explanation - “jitendra23” word is matched and it is also the first position of the string .

Solution

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
int match(string st, string pat);
```

```
int main() {
```

```
    string st, pat;
```

```
    int status;
```

```
    getline(cin, st);
```

```
    getline(cin, pat);
```

```

    status = match(st, pat);

    if (status == -1) {

        cout << "Not Hacked" << endl;

    } else {

        cout << "Hacked " << status << endl;

    }

    return 0;

}

int match(string st, string pat) {

    int n = st.length();

    int m = pat.length();

    int i, j, count = 0, temp = 0;

    for (i = 0; i <= n - m; i++) {

        temp++;

        for (j = 0; j < m; j++) {

            if (st[i + j] == pat[j]) {

                count++;

            }

        }

        if (count == m) {

            return temp;

        }

        count = 0;

    }

    return -1;

}

```

Test Cases

<u>Input</u>	<u>Output</u>
jitendra23#kunal@32#rajesh_56#kartik_007 jitendra23	Hacked 1
jitus23#panj65#rajesh001#vishal001#vikas_23#rajat2 rajat2	Hacked 45
jitus23#panj65#rajesh001#vishal001#vikas_23#rajat2#kunal23 vishal003	Not Hacked
jitus23#panj65#rajesh001#vishal001#vikas_23#rajat2#kunal23 rajesh001	Hacked 16
whysoserious#joker09#batman23#superman99#spiderman3 spiderman3	Hacked 42
flash23#v3n0m#hk56#indian23#vbnm4	Hacked 15

hk56	
------	--

Section - 4 (2*10) Coding Questions 10 Marks

Q 1. Good numbers in the array.

You are given an `arr[]` consisting of `n` positive integers and you have to find the count of good integers in the array. Elements which are considered as the good number must follow 2 properties.

1. Element itself is a prime number.
2. And all digits present in the element must be a prime number.

Input Format:

First line contains integer `N` as the size of the array.
Next line contains `N` integers as elements of array.

Output Format:

Print the total count of good numbers present in the given array.

Constraints:

$$0 < N < 1000$$

$$0 < arr[i] \leq 100000$$

Input	Output
-------	--------

8 2 3 6 11 22 23 27 16	3
1 147	0

Explanation:

In the first example 2, 3, 23 are good numbers because they are prime and all the digits they consist of are also prime numbers.

In Second example 147 is not a good number , because it doesn't follow the second condition , 4 is not a prime number.

Solution:

```
#include <iostream>
```

```
using namespace std;
```

```
bool isPrime(int n) {
```

```
    if (n <= 1)
```

```
        return false;
```

```
    if (n == 2 || n == 3)
```

```
        return true;
```

```
    if (n % 2 == 0 || n % 3 == 0)
```

```
        return false;
```

```
    for (int i = 5; i <= sqrt(n); i = i + 6)
```

```
        if (n % i == 0 || n % (i + 2) == 0)
```

```
        return false;
```

```
    return true;
}
```

```
bool isgoodnumber(int n ){
    if(isPrime(n) == false)return false;
    while(n>0){
        int x = n%10;
        if(isPrime(x) == false)return false;
        n = n/10;
    }
    return true;
}
```

```
int main() {
    int m;
    cin >> m;
    int arr[m];
    for (int i = 0; i < m; i++) {
        cin >> arr[i];
    }
    int count = 0;
    for(int i = 0; i < m; i++) {
        if(isgoodnumber(arr[i])){
            count ++;
        }
    }
}
```



```
cout << count << endl;
```

```
return 0;
```

```
}
```

Input	Output
10 2 3 6 11 22 23 27 16 5 147	4
10 2 3 6 110 22 253 27 16 5 147	3
8 2 3 5 2 3 22 10 12	5
6 3 4 5 6 7 22	3
15 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15	4
16 23 29 17 37 52 47 2 3 5 7 6 8 22 234 42 21	6
11 23 29 257 17 37 52 47 2 3 5 11	6

1 257	1
3 257 2 4	2
6 11 12 13 14 15 16	0

Q 2. Buy and sell Stocks :

Given an array `prices[]` of length `N`, representing the prices of the stocks on different days, the task is to find the maximum profit possible by buying and selling the stocks on different days when at most one transaction is allowed. Here one transaction means 1 buy + 1 Sell.

Note: Stock must be bought before being sold.

Input Format

First line - Take size of array `N`.

Second line - Take `N` integer value to store values of stock per day in an array.

Output Format

Print the maximum profit to buy and sell stock 1 time.

Constraints

$$1 \leq \text{arr.size()} \leq 10^5$$

$$0 \leq \text{arr}[i] \leq 10^3$$

Input	Output
9 7 1 5 2 9 4 11 8 0	10
17 3 3 3 3 2 1	0

Explanation - we will get maximum profit when we buy on price 0 & sell it on price 11.

Solution

```
#include<iostream>

using namespace std;

int maxSubarraySum(int prices[], int n){
    int res = 0;

    // Explore all possible ways to buy and sell stock
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            res = max(res, prices[j] - prices[i]);
        }
    }

    return res;
}

int main(){
    int n;
```

```
cin>>n;
```

```
int arr[n];
```

```
for(int i=0; i<n; i++){
```

```
    cin>>arr[i];
```

```
}
```

```
cout << maxSubarraySum(arr, n) << endl;
```

```
return 0;
```

```
}
```

Test Cases

<u>Input</u>	<u>Output</u>
10 23 17 45 3 28 19 41 36 25	38
6 3 3 3 3 3 3	0
9 1 2 3 4 5 6 7 8 9	8
6 6 5 4 3 2 1	0

6 7 1 5 3 6 4	5
6 1 11 10 110 111 1110	1109
3 19 0 19	19
8 33 352 323 56 79 96 339 89	319
12 342 3235 87 7678 7698 876 876 978 79 97 9879 8789	9800
15 8746 5143 6134 4664 4361 4100 2432 3523 3253 4256 9879 6757 2312 5435 8968	7447

Section - 3 (4*5)

Coding Questions 5 Marks

Q1 Write a program to find the sum of the digits of a given number N.

Input Format

An integer

Output Format

Print the sum of integer

Constraints

$0 < N < 100000$

Input	Output
62378	26

Solution

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    int num, sum = 0, remainder;
```

```
    cin >> num;
```

```
    while (num > 0) {
```

```
        remainder = num % 10;
```

```
sum += remainder;
```

```
num /= 10;
```

```
}
```

```
cout << sum << endl;
```

```
return 0;
```

```
}
```

Test Cases

Input	Output
15054	15
98827	34
74011	13
18975	30
8203	13

Ques 2:- TOPIC: DYNAMIC MEMORY ALLOCATION

Write a program that takes the size of an array as input, dynamically allocates memory for an integer array of that size, allows the user to input elements into the array, displays the elements, finally prints the sum of elements present in the array and deallocates the dynamically allocated memory.

Input Format:

First line : containing an integer N (size of array)

Second line : containing N integers (elements of the array)

Output Format:

Sum of all the elements in the array.

Constraints:

$N \leq 10000000$

$A[i] \leq 1000000000$

Input	Output
7 5249 73 3658 8930 1272 7544 878	27604

Explanation:

Sum of all the elements in the array for this case is : $5249 + 73 + 3658$
 $+ 8930 + 1272 + 7544 + 878 = 27604$

Solution:

```
#include<iostream>
```

```
using namespace std;
```

```
int main(){
```

```
    int n;
```

```
    cin>>n;
```

```
    int *arr = new int[n];
```

```
    for(int i=0;i<n;i++){
```

```
        cin>>arr[i];
```

```
    }
```

```
    int sum = 0;
```

```
    for(int i=0;i<n;i++){
```

```
        sum += arr[i];
```

```
    }
```

```
    cout<< sum <<endl;
```

```
    delete[] arr;
```

```
    return 0;
```

```
}
```

Test Cases

<u>Input</u>	<u>Output</u>
7 5249 73 3658 8930 1272 7544 878	27604
101 75249 50073 143658 108930 11272 27544 50878 177923 37709 164440 38165 84492 43042 7987 122503 82327 131729 178840 142612 144303 33169 17709 97157 129560 170933 93099 80278 151816 195335 99097 7826 13512 129267 123810 77633 180979 179149 36579 158821 111967 10672 101393 19336 125485 91745 25228 94091 40194 186357 35001 121153 16708 157944 115668 171490 188124 102196 109530 60903 127722 54666 128549 118024 197801 6853 140977 47408 108228 174933 60298 138981 28635 8013 23865 189814 139063 154536 139425 1669 124115 20094 75629 46501 146517 134195 30105 10404 119451 194298 182188	9734408

21123 199505 46882 6752 131566 136716 130337 154438 53144 51501 44897	
--	--

401

249 73 158 430 272 44
378 423 209 440 165 492
42 487 3 327 229 340 112
303 169 209 157 60 433
99 278 316 335 97 326 12
267 310 133 479 149 79
321 467 172 393 336 485
245 228 91 194 357 1 153
208 444 168 490 124 196
30 403 222 166 49 24 301
353 477 408 228 433 298
481 135 13 365 314 63 36
425 169 115 94 129 1 17
195 105 404 451 298 188
123 5 382 252 66 216 337
438 144 1 397 371 328
137 358 177 397 294 404
109 231 245 175 135 298
142 399 468 412 260 57
94 8 395 468 113 30 6
462 442 365 82 352 267
321 195 212 171 401 90
331 238 57 116 290 140
179 335 6 472 98 95 319
454 223 289 260 405 126
123 6 313 270 238 94 220
344 366 34 226 394 363
238 344 90 50 159 123
447 385 217 272 39 247
385 496 385 123 420 144
468 235 415 125 34 42 11
179 152 244 295 318 396
192 315 491 33 169 397
53 47 325 210 162 211
308 377 261 125 335 368
495 276 267 439 374 331
56 301 372 60 94 484 255
289 407 15 193 269 180
455 355 6 463 2 176 108
249 331 344 138 308 497

97029

151 349 203 231 31 14 419 275 9 180 429 223 54 260 237 45 317 25 200 256 64 97 148 204 50 150 476 412 54 297 4 381 248 65 378 199 209 129 53 483 447 107 273 322 305 176 53 224 130 366 400 444 370 285 16 398 155 133 57 76 178 266 357 211 378 114 319 237 133 91 220 262 133 197 31 88 89 373 377 304 358 200 254 460 415 447 480 230 455 46 107 238 0 426 35 357 113 114 93 360 354 418 357 85 229 15 63 102 417 143 419 467 247 269 395 303 473 103 248 385 158 459 406 430 324 132 473 103 397 420 450 231 480 203 400 20 33 258 3 176 450 434 280 379 332 74 49 42	
--	--

108

249 73 158 430 272 44
378 423 209 440 165 492
42 487 3 327 229 340 112
303 169 209 157 60 433
99 278 316 335 97 326 12
267 310 133 479 149 79
321 467 172 393 336 485
245 228 91 194 357 1 153
208 444 168 490 124 196
30 403 222 166 49 24 301
353 477 408 228 433 298
481 135 13 365 314 63 36
425 169 115 94 129 1 17
195 105 404 451 298 188
123 5 382 252 66 216 337
438 144 1 397 371 328
137 358 177 397 294

25970

651

249 73 158 430 272 44
378 423 209 440 165 492
42 487 3 327 229 340 112
303 169 209 157 60 433
99 278 316 335 97 326 12
267 310 133 479 149 79
321 467 172 393 336 485
245 228 91 194 357 1 153
208 444 168 490 124 196
30 403 222 166 49 24 301
353 477 408 228 433 298
481 135 13 365 314 63 36
425 169 115 94 129 1 17
195 105 404 451 298 188
123 5 382 252 66 216 337
438 144 1 397 371 328
137 358 177 397 294 404
109 231 245 175 135 298
142 399 468 412 260 57
94 8 395 468 113 30 6
462 442 365 82 352 267
321 195 212 171 401 90
331 238 57 116 290 140
179 335 6 472 98 95 319
454 223 289 260 405 126
123 6 313 270 238 94 220
344 366 34 226 394 363
238 344 90 50 159 123
447 385 217 272 39 247
385 496 385 123 420 144
468 235 415 125 34 42 11
179 152 244 295 318 396
192 315 491 33 169 397
53 47 325 210 162 211
308 377 261 125 335 368
495 276 267 439 374 331
56 301 372 60 94 484 255
289 407 15 193 269 180
455 355 6 463 2 176 108
249 331 344 138 308 497

155051

151 349 203 231 31 14
419 275 9 180 429 223 54
260 237 45 317 25 200
256 64 97 148 204 50 150
476 412 54 297 4 381 248
65 378 199 209 129 53
483 447 107 273 322 305
176 53 224 130 366 400
444 370 285 16 398 155
133 57 76 178 266 357
211 378 114 319 237 133
91 220 262 133 197 31 88
89 373 377 304 358 200
254 460 415 447 480 230
455 46 107 238 0 426 35
357 113 114 93 360 354
418 357 85 229 15 63 102
417 143 419 467 247 269
395 303 473 103 248 385
158 459 406 430 324 132
473 103 397 420 450 231
480 203 400 20 33 258 3
176 450 434 280 379 332
74 49 42 249 271 310 379
483 470 40 223 150 471
229 318 246 299 230 121
276 124 244 458 196 271
64 60 98 251 440 3 51
301 205 80 418 274 149
413 320 25 12 283 288
117 8 50 324 195 257 11
190 166 410 93 53 65 460
242 403 352 307 141 410
200 299 127 171 287 414
125 141 17 348 342 315
474 445 373 220 411 239
54 305 429 253 189 166
356 304 360 398 92 220
116 80 367 352 439 198
401 311 116 391 171 228
171 161 45 120 240 158
453 127 350 211 63 94 33
227 327 295 31 442 16 6

15 424 101 97 161 103	
324 108 436 281 15 210	
216 405 473 281 220 291	
169 365 327 36 228 125	
384 167 449 476 46 76	
166 467 20 384 491 210	
258 211 244 18 218 434	
325 267 189 318 14 325	
418 28 29 34 227 240 254	
40 396 435 83 48 365 11	
252 64 376 237 175 245	
210 415 192 210 247 37	
299 447 215 193 79 129	
164 479 1 25 289 458 33	
300 89 170 217 346 38 43	
338 436 321 219 496 247	
288 359 487 35 235 412	
184 489 284 234 167 19	
115 45 476 161 325 319	
131 403 198 298 139 4 3	
179 265	

Ques 3 -Maximum Among Left:

Given a vector V of n elements. We need to print the list of indices such that V_i is strictly greater than all the elements from 0 to $i-1$.

Note: The resultant list may be empty (means you have nothing to print).

Input Format

First line - Take size of vector N.

Second line - Take N integer values to store them in a vector.

Output Format

Print the list of all indices whose elements are maximum among the left.

Constraints

$$1 \leq \text{arr.size()} \leq 10^5$$

$$-10^3 \leq \text{arr}[i] \leq 10^3$$

Input	Output
5 4 3 5 2 6	2 4

Explanation - The given elements are {4 3 5 2 6}. 5 is greater than all elements to the left (4 and 3). 6 is greater than all elements to the left. So the answer list contains indices of 5 and 6 {2 4}.

.

Solution

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;

vector<int> check(vector<int>&v){

    int max = v[0];

    vector<int> ans;

    for(int i = 1;i<v.size();i++){

        if(v[i]>max){

            max=v[i];

            ans.push_back(i);

        }

    }

    return ans;

}

int main(){

    int n;

    cin>>n;

    vector<int>ans(n);

    for(int i =0; i<n; i++){

        cin>>ans[i];

    }

    vector<int> arr = check(ans);

    for(int i =0; i<arr.size(); i++){

        cout << arr[i]<< " ";

    }

    cout << endl;

    return 0;
```

}

Test Cases

<u>Input</u>	<u>Output</u>
5 1 2 3 -2 5	1 2 4
12 2 1 -2 55 -13 21 18 10 -27 17 9 2	3
12 -10 -20 -30 -40 -50 -60 -70 -80 -90 -100 -110 -120	
15 10 20 30 40 50 60 70 80 90 100 110 120 130 140 150	1 2 3 4 5 6 7 8 9 10 11 12 13 14
19 15 -12 8 -17 19 20 -16 14 13 -11 10 9 -8 7 6 -5 4 3 -2 1	4 5
16	1 3 5 7 9 11 13 15

-100 200 -300 400 -500 600 -700 800 - 900 1000 -1100 1200 -1300 1400 -1500 1600	
---	--

Ques 4 -

Take as input S, a string. Write a program that inserts between each pair of characters the difference between their ascii codes and print the ans.

Input Format

Take an input string from user.

Output Format

Print the string with the difference between the adjacent characters in the string

Constraints

Length of String should be between 2 to 1000.

Input	Output
acb	a2c-1b

Explanation

For the sample case, the Ascii code of a=97 and c=99 ,the difference between c and a is 2. Similarly ,the Ascii code of b=98 and c=99 and their difference is -1. So the ans is a2c-1b.

Solution

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
string insertAsciiDifferences(string s) {
```

```
    string result = "";
```

```
    for (int i = 0; i < s.length() - 1; i++) {
```

```
        result += s[i];
```

```
        result += to_string(int(s[i + 1]) - int(s[i]));
```

```
    }
```

```
    result += s[s.length() - 1];
```

```
    return result;
```

```
}
```

```
int main() {
```

```
    string s;
```

```
    getline(cin, s);
```

```
cout << insertAsciiDifferences(s) << endl;
```

```
return 0;
```

```
}
```

Test Cases

<u>Input</u>	<u>Output</u>
acb	a2c-1b
abcd	a1b1c1d
xyza	x1y1z-25a
hello	h-3e7l0l3o
abcde	a1b1c1d1e

Section - 4 (2*10) Coding Questions 10 Marks

Q 1. Create a C++ class Complex that represents complex numbers (real + imag i). Overload the + operator to allow the addition of two complex numbers. The class should have the following specifications:

- A. Private member variables for the real part (real) and the imaginary part (imag) of the complex number .
- B. A parameterized constructor to initialise the complex number.
- C. Overload the + operator to add two complex numbers.
- D. Overload the * operator to multiply two complex numbers
- E. Implement a method display to display the complex number.

Your output should contain the addition and multiplication of two complex numbers whose real and imaginary parts are given as input by the user.

Input Format:

First line: 2 integers - real and imaginary part of first complex number

Second line: 2 integers - real and imaginary part of second complex number

Output Format:

First line: Addition of two complex numbers

Second line: Multiplication of two complex numbers

Constraints:

$1 \leq \text{real}, \text{imag} \leq 1000$

Input	Output
2 3	6 + 8 i
4 5	-7 + 22 i

Explanation:

Addition of $(2+3i)$ and $(4+5i)$ is $(6 + 8 i)$

Multiplication of $(2+3i)$ and $(4+5i)$ is $(-7 + 22 i)$

Solution:

```
#include <iostream>
```

```
using namespace std;
```

```
class Complex {
```

```
private:
```

```
    int real;
```

```
    int imag;
```

```
public:
```

```
    Complex(int r, int i){
```

```
        real = r;
```

```
        imag = i;
```

```
}
```

```
Complex operator+(const Complex& other) const {  
    return Complex(real + other.real, imag + other.imag);  
}
```

```
Complex operator*(const Complex& other) const{  
    return Complex(real*other.real - imag*other.imag,  
real*other.imag+imag*other.real);  
}
```

```
void display() const {  
    cout << real << " + " << imag << " i" << endl;  
}  
};
```

```
int main() {  
    int a,b,c,d;  
    cin >> a >> b >> c >> d;  
    Complex c1(a,b);  
    Complex c2(c,d);
```

```
    Complex addition = c1 + c2;
```

```
    Complex multiplication = c1 * c2;
```

```
addition.display();
```

```
multiplication.display();
```

```
return 0;
```

```
}
```

Test Cases:

Input	Output
2 3	$6 + 8 i$
4 5	$-7 + 22 i$
5 7	$6 + 16 i$
1 9	$-58 + 52 i$
8 1	$11 + 8 i$
3 7	$17 + 59 i$

4 1 5 5	$9 + 6i$ $15 + 25i$
1 3 1000 44	$1001 + 47i$ $868 + 3044i$

Q 2. Print elements according to pattern similar to number 7 :

Given a square matrix `mat[][]`, and you have to traverse the input matrix and print elements according to a specific pattern which is similar to integer 7.

Note -> you can take reference to sample input and output for better understanding which type of pattern you have to print using array elements.

Input Format

First line - Take size of array N.

Second line - Take N*N integer value to store them in a 2D array or square matrix.

Output Format

Print all the elements in specific order which came under a pattern similar to integer 7.

Output elements should print in a single line and space separated.

Constraints

$$3 \leq N \leq 11$$

$$0 \leq \text{arr}[i] \leq 10^3$$

Input	Output
3 7 1 5 2 9 4 11 8 0	7 1 5 9 11
5 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25	1 2 3 4 5 9 13 17 21

Explanation - in the first example we have to print all elements present on the pattern of 7. And it will print the first row left to right then it will print right diagonal from top to bottom.

In the second example when we print all elements present on the pattern of 7 are 1 2 3 4 5 then right diagonal 9 13 17 21.

Solution

```
#include <iostream>

#include <vector>

#include <algorithm>

using namespace std;

void print(vector<vector<int> >& mat) {

    int n = mat.size();

    vector<int>ans;

    int count =0;

    for(int i =0 ; i<n; i++){

        ans.push_back(mat[0][i]);

        count++;

    }

    for(int i =1; i<n; i++){

        for(int j =0; j<n; j++){

            if(i+j == n-1){

                ans.push_back(mat[i][j]);

                count++;

            }

        }

    }

}

for(int i =0; i<count; i++ ){
```

```

        cout << ans[i]<< " ";
    }

    cout << endl;
}

int main(){

    int N;

    cin>>N;

    vector<vector<int> >ans(N, vector<int>(N));

    for(int i = 0; i < N; i++) {

        for(int j = 0; j < N; j++) {

            cin>>ans[i][j];

        }

    }

    print(ans);

    return 0;

}

```

Test Cases -

<u>Input</u>	<u>Output</u>
3 17 11 9 3 3 3 3 2 1	17 11 9 3 3

5 1 2 3 4 5 6 7 8 9 1 9 12 1 14 15 16 1 18 19 20 2 22 2 24 25	1 2 3 4 5 9 1 1 2
3 1 2 3 4 5 6 7 8 9	1 2 3 5 7
5 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25	1 2 3 4 5 9 13 17 21
3 9 8 7 6 5 4 3 2 1	9 8 7 5 3

3 1 2 3 4 1 2 3 4 1	1 2 3 1 3
3 9 9 9 9 9 9 9 9 9	9 9 9 9 9 9 9 9
3 12 10 2 3 3 4 2 8 3	12 10 2 3 2
3 1 2 4 3 6 9 5 10 15	1 2 4 6 5

5

1 2 3 4 5

1 2 3 4 5

1 2 3 4 5

1 2 3 4 5

1 2 3 4 5

1 2 3 4 5 4 3 2 1

Section - 3 (4*5)

Coding Questions 5 Marks

Q1 Swap first and last character of each word in a string.

Write a program which given a string, swap first and last character of each word in it.

Complete the function `swapFirstLastChar()` that accepts a multiword string and swaps the first and last character of every word in it.

Note: Every two adjacent words in the given string can be separated by one or more than one space character

Input Format

A single string containing multiple words separated by exactly one space character.

Each word in the string will contain at least one character.

The input string can have multiple words, and words can have varying lengths.

Output Format

A single string where the first and last characters of each word in the input string have been swapped. The words should be separated by exactly one space character.

Constraints

The input string will only contain lowercase and uppercase alphabetic characters and spaces.

$1 \leq s.length \leq 10^4$

Each word in the string will have at least one character.

Adjacent words can be separated by one or more than one space character.

Input	Output
Code Quotient	eodC tuotienQ

Solution

```
#include <iostream>
```

```
#include <string>
```

```
#include <sstream>
```

```
#include <vector>
```

```
using namespace std;
```

```
string swapFirstLastChar(string str) {
```

```
    stringstream ss(str);
```

```
    string word, ans;
```

```
    vector<string> words;
```

```
    while (ss >> word) {
```

```

        words.push_back(word);
    }

    for (string word : words) {
        if (word.length() > 1) {
            char first = word[0];
            char last = word[word.length() - 1];
            word = last + word.substr(1, word.length() - 2) + first;
        }
        ans += word + " ";
    }

    return ans.substr(0, ans.length() - 1);
}

int main() {
    string userInput;
    getline(cin, userInput);

    cout << swapFirstLastChar(userInput) << endl;

    return 0;
}

```

Test Cases

Input	Output
-------	--------

Code Quotient	eodC tuotienQ
Get better at coding	teG retteb ta godinc
a b c d e	a b c d e
swap	pwas
hello world	oellh dorlw

Ques 2 You are given two strings word1 and word2. Merge the strings by adding letters in alternating order, starting with word1. If a string is longer than the other, append the additional letters onto the end of the merged string.

Return the merged string.

Input Format

First line - Take first string.

Second line - Take second string.

Output Format

Merged the output string and print the output string.

Constraints

$1 \leq \text{string length} \leq 100$

Input	Output
abc pqr	apbqcr

Explanation - In the first iteration, it appends "a" from **word1** to the merged string. In the second iteration, it appends "p" from **word2** to the merged string. This process continues, alternating between characters from **word1** and **word2**, until both strings have been exhausted. The final merged string is "apbqcr", which is the expected result of merging the two input strings in alternating order.

Solution

```
#include <iostream>
```

```
#include <cstring>
```

```
using namespace std;
```

```
char* mergeAlternately(char* word1, char* word2) {
```

```
    int len1 = strlen(word1);
```

```
    int len2 = strlen(word2);
```

```
    int len = len1 + len2 + 1;
```

```
    char* result = new char[len];
```

```
    int i = 0, j = 0, k = 0;
```

```
    while (i < len1 || j < len2) {
```

```

        if (i < len1) {

            result[k++] = word1[i++];

        }

        if (j < len2) {

            result[k++] = word2[j++];

        }

    }

    result[k] = '\0';

    return result;

}

int main() {

    char s1[100], s2[100];

    cin >> s1 >> s2;

    char* merged = mergeAlternately(s1, s2);

    cout << merged << endl;

    delete[] merged;

    return 0;

}

```

Test Cases

<u>Input</u>	<u>Output</u>

ab pqrs	apbqrs
abcd pq	apbqcd
abcdef pqrs	apbqcrdsef
a pqrs	apqrs
rajesh kumar	rkaujmeasrh
never giveup	ngeivveerup

Ques 3 - Maximum power by single character :

Given a string and each character of string associated with an integer value defining power, find the maximum power generated by a single character .

Each character associated with integers values are as follows : a = 52, b = 50 , c = 48 , d = 46, e = 44 , , x = 4 , y =2, z = 0

Note- when a character occurs in string it will give value/ power equal to its associative value given in above line.

(refer example for better understanding)

Input Format

Take a string from user only lowercase letters

Output Format

You have to print the maximum output that can be generated by a single char.

Constraints

Input	Output
chitkarauniversity	156

Explanation -

Solution

```
#include<iostream>
```

```
using namespace std;
```

```
int maxSubarraySum(int arr[], int n){
```

```
    int maxending =0;
```

```

    int max_so_far = arr[0];

    int curr_max = arr[0];

    for(int i=1; i<n; i++){

        curr_max = max(arr[i], curr_max+arr[i]);

        max_so_far = max(max_so_far, curr_max);

    }

    return max_so_far;

}

int main(){

    int n;

    cin>>n;

    int arr[n];

    for(int i=0; i<n; i++){

        cin>>arr[i];

    }

    cout << maxSubarraySum(arr, n) << endl;

    return 0;

}

```

Test Cases

<u>Input</u>	<u>Output</u>
chitkarauniversity	156

arivlambaisthebestprofessorever	220
orbhaikhaalchalhteresundemitarrrr	260
mujhenhiptaatukudhsochhehehehe	304
karthikbhaiyaankitkaphonekyunhiuthateh	312
yhanindaarhihorpaperbananapaddrh ahuppersemailpemail	572

Ques 4 -Prime At Prime Index:

You are given vector arr from the user as input. Print all the elements present in the vector containing prime numbers that are at prime index(1-indexing).

Note: 1- indexing means consider the 1st element as on index 1.

Input Format

First line - Take size of vector N.

Second line - Take N integer values to store them in a vector.

Output Format

Print all the elements which are prime and also present at the prime index (1 - indexing).

Constraints

$1 \leq \text{arr.size()} \leq 10^5$

$1 \leq \text{arr}[i] \leq 10^5$

Input	Output
5 3 5 2 4 8	5 2

Explanation - Only 5 and 2 are the numbers that are prime and are present at prime position (2 and 3 respectively).

Solution

```
#include<iostream>
```

```
#include<vector>
```

```
using namespace std;
```

```
bool isPrime(int num)
```

```
{
```

```
    if(num<2)
```

```
        return false;
```

```
    for(int i=2;i*i<=num;i++)
```

```
if(num%i==0)
```

```
return false;
```

```
return true;
```

```
}
```

```
vector<int> check(vector<int>v)
```

```
{
```

```
// Your code here
```

```
vector<int> v1;
```

```
for(int i=1;i<=v.size();i++)
```

```
if(isPrime(i+1) && isPrime(v[i]))
```

```
v1.push_back(v[i]);
```

```
return v1;
```

```
}
```

```
int main(){
```

```
int n;
```

```
cin>>n;
```

```
vector<int>arr(n);
```

```
for(int i=0; i<n; i++){
```

```
cin>>arr[i];
```

```
}
```

```
vector<int>ans = check(arr);
```

```
for(int i =0; i<ans.size(); i++){
```

```
cout << ans[i]<< " ";
```

```

    }

    cout << endl;

    return 0;

}

```

Test Cases

<u>Input</u>	<u>Output</u>
5 3 5 2 4 8	5 2
5 5 2 3 5 7	2 3 7
5 5 2 3 4 5	2 3 5
7 5 11 13 17 19 1 2	11 13 19 2
10 5 11 13 17 19 1 2 8 9 11	11 13 19 2

15	2 3 5 7 11 13
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15	

Section - 4 (2*10) Coding Questions 10 Marks

Q 1.

You are given a base class Person with the following attributes and methods:

Attributes:

(i) Name (string)

(ii) Age (int)

Methods:

display : displays the name and age of the person on separate lines.

Your task is to create a derived class Employee that inherits from the Person class. The Employee class should have the following additional attributes and methods:

- employeeId (a unique identifier for each employee, an integer)
- salary (the employee's salary, a double)
- display method (overriding the display method from the base class) that displays the employee's name, age, employeeId, and salary.

Write the code for the Employee class that inherits from the Person class, and ensure the display method is correctly overridden to include the new attributes.

You will be given the name(string), age(int), employeeId(int), salary(double) as input.

Finally display the employee's details using the display function inside Employee class.

Input Format:

4 space separated values: name(string), age(int), employeeId(int), salary(int)

Output Format:

Display the employee's details. (as shown in sample test case)

Sample Test Case:

Input	Output
Amol 27 234 100000	Name: Amol Age: 27 Employee ID: 234 Salary: 100000

Solution:

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
class Person {
```

```
protected:
```

```
    string name;
```

```
    int age;
```

```
public:
```

```
    Person(const string& name, int age) : name(name), age(age) {}
```

```
    void display() {
```

```
        cout << "Name: " << name << "\n";
```

```
        cout << "Age: " << age << "\n";
```

```
    }
```

```
};
```

```
class Employee : public Person {
```

```
private:
```

```
    int employeeId;
```

```
    double salary;
```

```
public:
```

```
    Employee(const string& name, int age, int employeeId, double salary)
```

```
        : Person(name, age), employeeId(employeeId), salary(salary) {}
```

```
    void display() {
```

```
        Person::display();
```

```
        cout << "Employee ID: " << employeeId << "\n";
```

```
        cout << "Salary: " << salary << "\n";
```

```
    }
```

```
};
```

```
int main() {
```

```
    string name;
```

```
    int age, employeeId;
```

```
    double salary;
```

```
    cin >> name >> age >> employeeId >> salary;
```

```
    Employee employee(name, age, employeeId, salary);
```

```
    employee.display();
```

```
    return 0;
```

```
}
```

Test Cases:

Input	Output

Amol 27 234 100000	Name: Amol Age: 27 Employee ID: 234 Salary: 100000
Alice 28 101 55000	Name: Alice Age: 28 Employee ID: 101 Salary: 55000
Bob 35 102 12000	Name: Bob Age: 35 Employee ID: 102 Salary: 12000
Carol 22 103 67000	Name: Carol Age: 22 Employee ID: 103 Salary: 67000

David 40 190 17800	Name: David Age: 40 Employee ID: 190 Salary: 17800
--------------------	---

Q 2. Print elements according to pattern similar to number 2 :

Given a square matrix `mat[][]`, and you have to traverse the input matrix and print elements according to a specific pattern which is similar to integer 2.

Note -> you can take reference to sample input and output for better understanding which type of pattern you have to print using array elements.

Input Format

First line - Take size of array N.

Second line - Take N*N integer value to store them in a 2D array or square matrix.

Output Format

Print all the elements in specific order which came under a pattern similar to integer 2.

Output elements should print in single line and space separated.

Constraints

$$3 \leq N \leq 11$$

$$0 \leq \text{arr}[i] \leq 10^3$$

Input	Output
-------	--------

3 7 1 5 2 9 4 11 8 0	7 1 5 4 9 2 11 8 0
5 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25	1 2 3 4 5 10 15 14 13 12 11 16 21 22 23 24 25

Explanation - in the first example we have to print all elements present on the pattern of 2. And it will print the first row left to right , then the last row (top+1) to (1mid -1) index, then middle row in opposite direction , and so on.

In the second example when we print all elements present on the pattern of 2 are 1 2 3 4 5 then 10 then 15 14 13 12 11 then 16 then 21 22 23 24 25.

Solution

```
#include <iostream>
```

```
#include <vector>
```

```
#include <algorithm>
```

```
using namespace std;
```

```
void print(vector<vector<int> >& mat) {
```

```
    int n = mat.size();
```

```
    vector<int>ans;
```

```
    int count =0;
```

```
    for(int i =0; i<n; i++){
```

```
        ans.push_back(mat[0][i]);
```

```
        count++;
```

```
    }
```

```
    for(int i =1; i<(n/2); i++){
```

```
        ans.push_back(mat[i][n-1]);
```

```
        count++;
```

```
    }
```

```
    for(int i =n-1; i>=0; i--){
```

```
        ans.push_back(mat[n/2][i]);
```

```
        count++;
```

```
    }
```

```
    for(int i = (n/2)+1; i<n-1; i++){
```

```
        ans.push_back(mat[i][0]);
```

```
        count++;
```

```
    }
```

```
    for(int i =0; i<n; i++){
```

```
        ans.push_back(mat[n-1][i]);
```

```
        count++;
```

```
    }
```

```
    for(int i =0; i<count; i++ ){
```

```

        cout << ans[i]<< " ";
    }

    cout << endl;
}

int main(){
    int N;

    cin>>N;

    vector<vector<int> >ans(N, vector<int>(N));

    for(int i = 0; i < N; i++) {
        for(int j = 0; j < N; j++) {
            cin>>ans[i][j];
        }
    }

    print(ans);

    return 0;
}

```

Test Cases -

<u>Input</u>	<u>Output</u>
3 17 11 9 3 3 3 3 2 1	17 11 9 3 3 3 3 2 1

5 1 2 3 4 5 6 7 8 9 1 9 12 1 14 15 16 1 18 19 20 2 22 2 24 25	1 2 3 4 5 1 15 14 1 12 9 16 2 22 2 24 25
3 1 2 3 4 5 6 7 8 9	1 2 3 6 5 4 7 8 9
5 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25	1 2 3 4 5 10 15 14 13 12 11 16 21 22 23 24 25
3 9 8 7 6 5 4 3 2 1	9 8 7 4 5 6 3 2 1

3 1 2 3 4 1 2 3 4 1	1 2 3 2 1 4 3 4 1
3 9 9 9 9 9 9 9 9 9	9 9 9 9 9 9 9 9
3 12 10 2 3 3 4 2 8 3	12 10 2 4 3 3 2 8 3
3 1 2 4 3 6 9 5 10 15	1 2 4 9 6 3 5 10 15
5 1 2 3 4 5	1 2 3 4 5 5 5 4 3 2 1 1 1 2 3 4 5

1 2 3 4 5	
1 2 3 4 5	
1 2 3 4 5	
1 2 3 4 5	

Section - 3 (4*5)

Coding Questions 5 Marks

Q1 .Given a string `s` consisting of words and spaces, return the length of the last word in the string.

A word is a maximal substring consisting of non-space characters only.

Input Format

A single string containing multiple words separated by exactly one space character.

Each word in the string will contain at least one character.

The input string can have multiple words, and words can have varying lengths.

Output Format

Return the length of the last word of the string.

Constraints

- $1 \leq s.length \leq 104$
- `s` consists of only English letters and spaces ' '.
- There will be at least one word in `s`.

Input	Output
Hello World	5

Explanation: The last word is "World" with length 5.

Solution

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
int lengthOfLastWord(string s) {
```

```
    int count = 0;
```

```
    bool inWord = false;
```

```
    for (int i = s.length() - 1; i >= 0; i--) {
```

```
        if (isspace(s[i])) {
```

```
            if (inWord) {
```

```
                break;
```

```
            }
```

```
        } else {
```

```
            count++;
```

```
            inWord = true;
```

```
        }
```

```
}
```

```
    return count;
```

```
}
```

```
int main() {
```

```
    string s;
```

```
    getline(cin, s);
```

```
    int length = lengthOfLastWord(s);
```

```
    cout << length << endl;
```

```
    return 0;
```

```
}
```

Test Cases

Input	Output
Hello World	5
fly me to the moon	4
luffy is still joyboy	6
singleWord	10
a b c d e f g h i j k	1

Ques 2 - Write a code to calculate the last 3 digits of the Nth term of the fibonacci series.

Fibonacci Series: 0 1 1 2 3 5 8 13

The Fibonacci Series is the sequence where each number is the sum of the previous two numbers of the sequence. The first two numbers of the Fibonacci series are 0 and 1 and are used to generate the Fibonacci series.

Input Format

The one and only line of the input will contain N

Output Format

Print the number calculating the last 3 digits of the Nth term of the fibonacci series.

Constraints

$0 < N < 40$

Input	Output
15	377

Explanation - 15 th fibonacci is 377 so last three digits of 377 is 377.

Solution

```
#include <iostream>
```

```
using namespace std;
```

```
int fibonacci(int n) {
```

```
    if (n == 1) {
```

```
        return 0;
```

```
    } else if (n == 2) {
```

```
        return 1;
```

```
    } else {
```

```
        int a = 0, b = 1, c = 0;
```

```
        for (int i = 0; i < n - 2; i++) {
```

```
            c = (a + b) % 1000;
```

```
            a = b;
```

```
            b = c;
```

```
        }
```

```
        return c;
```

```
    }
```

```
}
```

```
int main() {
```

```
    int n;
```

```
    cin >> n;
```

```
    int ans = fibonacci(n);
```



```
cout << ans << endl;
```

```
return 0;
```

```
}
```

Test Cases

<u>Input</u>	<u>Output</u>
15	377
23	711
40	986
19	584
12	89
7	8

Ques 3 -Josephus problem:

A total of n people are standing in a circle, and you are one of them playing a game. Starting from a person, k persons will be counted in order along the circle, and the k th person will be killed. Then the next k persons will be counted along the circle, and again the k th person will be killed. This cycle will continue until only a single person is left in the circle.

If there are 5 people in the circle in the order A, B, C, D, and E, you will choose $k=3$. Starting from A, you will count A, B and C. C will be the 3rd person and will be killed. Then you will continue counting from D, E and then A. A will be third person and will be killed.

You will be given an array where the first element is the first person from whom the counting will start and the subsequent order of the people. You want to be the last one standing. Determine the index at which you should stand to survive the game

Input Format

First line - take 2 integers n and k .

(n represent number of position and k represent the next person to be killed)

Output Format

Print an integer denoting safe position.

Input	Output
3 2	3

Explanation - There are 3 persons so skipping 1 person i.e 1st person 2nd person will be killed. Thus the safe position is 3.

Solution-

```
#include <iostream>
```

```
#include <vector>
```

```
#include <list>
```

```
using namespace std;
```

```
int josephus(int n, int k)
```

```
{
```

```
    int count1;
```

```
    list<int>ans;
```

```
    for(int i=0; i<n;i++){
```

```
        ans.push_back(i);
```

```
    }
```

```
    auto it = ans.begin();
```

```
    while(ans.size()>1){
```

```
        for(int count =1; count<k; count++){
```

```
            it++;
```

```
            if(it==ans.end())
```

```
                it = ans.begin();
```

```
        }
```

```
        it = ans.erase(it);
```

```
        if(it==ans.end())
```

```
            it = ans.begin();
```

```

    }

    count1= *it+1;

    return count1;

}

int main(){

    int n, k;

    cin>>n>>k;

    cout << josephus(n, k)<< endl;

    return 0;

}

```

Test Cases

<u>Input</u>	<u>Output</u>
11 4	9
100 3	91
892 34	181
423 4	142

112 32	102
24214 3	11197

Ques 4 -Winner of an election:

Given an array of n names arr of candidates in an election, where each name is a string of lowercase characters. A candidate name in the array represents a vote casted to the candidate. Print the name of the candidate that received the maximum count of votes.

Note: If there is a draw between two candidates, then print *lexicographically* smaller name.

Input Format

First line - Take the size of the array of strings N.

Second line - Take N string value (name of candidate) to store them in an array.

Output Format

Print the name of the winner of the election and number of votes he/she gets..

Constraints

1. $2 \leq \text{size of vector} \leq 10^5$
2. All the characters in each string are in lowercase letters.

Input	Output
-------	--------

13	john 4
john johnny jackie johnny john	
jackie jamie jamie john johnny	
jamie johnny john	

Explanation - john has 4 votes casted for him, but so does johnny. john is lexicographically smaller, so we print john and the votes he received

Solution-

```
#include <iostream>
```

```
#include <vector>
```

```
#include <map>
```

```
#include <unordered_map>
```

```
using namespace std;
```

```
vector<string> winnerr(vector<string>&arr,int n)
```

```
{
```

```
    unordered_map<string, int>mp;
```

```
    for(int i=0; i<n; i++){
```

```
        mp[arr[i]]++;
```

```
    }
```

```
    string winner="";
```

```
    int maxvote=0;
```

```
    for(auto it=mp.begin(); it!=mp.end(); it++){
```

```

        if((it->second>maxvote)||((it->second==maxvote)&&(it->first<winner))){

            winner = it->first;

            maxvote = it->second;

        }

    }

    vector<string>ans;

    ans.push_back(winner);

    string x = to_string(maxvote);

    ans.push_back(x);

    return ans;

}

int main(){

    int n;

    cin>>n;

    vector<string>ans(n);

    for(int i=0; i<n; i++){

        cin>>ans[i];

    }

    vector<string> arr = winnerr(ans, n);

    for(int i=0; i<arr.size(); i++){

        cout << arr[i]<< " ";

    }

    cout << endl;

```

```
return 0;
```

```
}
```

Test Cases

<u>Input</u>	<u>Output</u>
12 john johnny jackie johnny john jackie jamie jamie john jamie johnny john johnny	john 4
3 andy blake clark	andy 1
22 john jane mike emily david john jane mike emily david john johnny jackie johnny john jackie jamie jamie john jamie johnny john johnny	john 6
10 john jane mike emily david john jane mike emily david	david 2
15 john jane mike emily david john jane mike emily david john jane mike emily david	david 3

45	emily 8
john jane mike emily david john jane mike emily david john jane mike emily david john jane mike emily david john jane mike emily david john jane mike emily david john jane mike emily david john jane mike emily	

Section - 4 (2*10) Coding Questions 10 Marks

Q 1. You are tasked with implementing a class called Counter to represent a simple counter(integer) initialised by 0 . The Counter class should provide the following functionality:

(i) Overload the ++ (pre increment) and -- (pre decrement) unary operators to increase and decrease the counter by 1, respectively.

(ii) Overload the + operator to allow adding an integer to the counter.

(iii) Overload the - operator to allow subtracting an integer from the counter.

You will be given 4 integers - x,y,n,m, you have to increment the value of counter x times (using ++ operator), decrement the value of counter y times (using -- operator) , increment the value of counter by n (using + operator), decrement the value of counter by m (using - operator). Finally print the value of the counter.

Input Format:

4 space separated integers: x,y,n and m.

Output Format

Single integer : value of counter

Constraint:

$0 \leq x,y,n,m \leq 100000$

Sample Test Case:

Input	Output
2 3 1 2	-2

Explanation:

Incrementing counter 2 times gives: 2 ($++0 = 1$ and $++1 = 2$)

Decrementing counter 3 times gives: -1 ($--2 = 1$, $-1 = 0$ and $-0 = -1$)

Adding 1 to counter gives: 0 ($-1+1 = 0$)

Subtracting 2 from counter gives us: -2($0-2 = -2$)

Hence print -2.

Solution:

```
#include <iostream>
```

```
using namespace std;
```

```
class Counter {
```

```
public:
```

```
    int count;
```

```
    Counter() : count(0) {}
```

```
    Counter(int initialCount) : count(initialCount) {}
```

```
    Counter& operator++() {
```

```
        ++count;
```

```
        return *this;
```

```
    }
```

```
    Counter& operator--() {
```

```
        --count;
```

```
        return *this;
```

```
    }
```

```
    Counter operator+(int n) {
```

```
        return Counter(count + n);
```

```
    }
```

```
    Counter operator-(int m) {
```

```
        return Counter(count - m);
```

```
}
```

```
};
```

```
int main() {
```

```
    int x, y, n, m;
```

```
    cin >> x >> y >> n >> m;
```

```
    Counter counter;
```

```
    for (int i = 0; i < x; ++i) {
```

```
        ++counter;
```

```
    }
```

```
    for (int i = 0; i < y; ++i) {
```

```
        --counter;
```

```
    }
```

```
    counter = counter + n;
```

```
    counter = counter - m;
```

```
    cout << counter.count << endl;
```

```
    return 0;
```

```
}
```

Test Cases:

Input	Output
2 3 1 2	-2
5 10 1000 23451	-22456
51239 10 1000 23451	28778
654 123 23 19	535
192 685 896 45	358

Q 2. Rotate the matrix clockwise in 90 :

Given a square matrix `mat[][]`, turn it by 90 degrees in an clockwise direction Note: it is a square matrix so , the number of rows must be equal to the number of columns.

Input Format

First line - Take size of array N.

Second line - Take N*N integer value to store them in a 2D array or square matrix.

Output Format

Print the rotated matrix in proper format.

First print 1st row in first line , second row in next line and so on. (refer sample output to print required output)

Constraints

$$1 \leq N \leq 7$$

$$0 \leq \text{arr}[i] \leq 10^3$$

Input	Output
3 7 1 5 2 9 4 11 8 0	11 2 7 8 9 1 0 4 5
3 17 11 9 3 3 3 3 2 1	3 3 17 2 3 11 1 3 9

Solution

```
// rotate the matrix clockwise.
```

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
void rotate90(vector<vector<int> >& mat) {
```

```
    int n = mat.size();
```

```
    vector<vector<int> > res(n, vector<int>(n));
```

```
    for (int i = 0; i < n; i++) {
```

```

        for (int j = 0; j < n; j++) {

            res[j][n - i - 1] = mat[i][j];

        }

    }

    mat = res;

}

int main(){

    int N;

    cin>>N;

    vector<vector<int> >ans(N, vector<int>(N));

    for(int i = 0; i < N; i++) {

        for(int j = 0; j < N; j++) {

            cin>>ans[i][j];

        }

    }

    rotate90(ans);

    for(int i = 0; i < N; i++) {

        for(int j = 0; j < N; j++) {

            cout<<ans[i][j]<<" ";

        }

        cout << endl;

    }

    return 0;

}

```

Test Cases

<u>Input</u>	<u>Output</u>
3 1 2 3 4 5 6 7 8 9	3 6 9 2 5 8 1 4 7
3 7 1 5 2 9 4 11 8 0	5 4 0 1 9 8 7 2 11
5 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25	5 10 15 20 25 4 9 14 19 24 3 8 13 18 23 2 7 12 17 22 1 6 11 16 21
4 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16	4 8 12 16 3 7 11 15 2 6 10 14 1 5 9 13
4 1 2 3 4 1 2 3 4 1 2 3 4 1 2 3 4	4 4 4 4 3 3 3 3 2 2 2 2 1 1 1 1

3 999999999	999 999 999
1 1	1
5 1234512345 1234512345123 45	55555 44444 33333 22222 11111
3 987654321	741 852 963
3 123412341	321 214 143

Section - 3 (4*5)

Coding Questions 5 Marks

Q1

Implement a program to convert a decimal number N to binary.

Input Format

The one and only line of the input will contain N

Output Format

The N lines of pattern

Constraints

$0 < N < 100000$

Input	Output
10	1010

Solution

```
#include <iostream>
```

```
using namespace std;
```

```
void decimalToBinary(int num) {
```

```
    int binary[32];
```

```
    int i = 0;
```

```

while (num > 0) {

    binary[i] = num % 2;

    num /= 2;

    i++;

}

for (int j = i - 1; j >= 0; j--) {

    cout << binary[j];

}

cout << endl;

}

int main() {

    int num;

    cin >> num;

    decimalToBinary(num);

    return 0;

}

```

Test Cases

Input	Output
-------	--------

69858	10001000011100010
53327	1101000001001111
66558	10000001111111110
23490	101101111000010
84454	10100100111100110

Ques 2 -

You are given an integer array nums. The unique elements of an array are the elements that appear exactly once in the array.

Return the sum of all the unique elements of nums.

Input Format

First line - An integer N

Second line - N elements of array

Output Format

An integer representing Sum of all unique elements.

Constraints

$1 \leq \text{arr.size()} \leq 100$

$1 \leq \text{arr}[i] \leq 10^2$

Input	Output
4 1 2 3 2	4

Explanation - Unique elements (1,3) occurs only one time . So , sum of unique elements are $1+3 = 4$

Solution

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
int sumOfUnique(vector<int>& nums) {
```

```
    int uni[101] = {0};
```

```
    int i, j;
```

```
    int sum = 0;
```

```
    for (i = 0; i < nums.size(); i++) {
```

```
        uni[nums[i]]++;
```

```

    }

    for (j = 0; j < 101; j++) {

        if (uni[j] == 1) {

            sum += j;

        }

    }

    return sum;

}

```

```

int main() {

    int n;

    cin >> n;

    vector<int> arr(n);

    for (int i = 0; i < n; i++) {

        cin >> arr[i];

    }

    cout << sumOfUnique(arr) << endl;

    return 0;

}

```

Test Cases

<u>Input</u>	<u>Output</u>

5 1 1 1 1 1	0
5 1 2 3 4 5	15
5 12 45 56 23 56	80
12 5 5 6 6 7 7 8 8 1 2 3 4	10
6 1 100 100 100 100 10	11
10 1 55 33 22 67 12 55 33 22 12	68

Ques 3 -

You are given a string title consisting of one or more words separated by a single space, where each word consists of English letters. Capitalize the string by changing the capitalization of each word such that:

If the length of the word is 1 or 2 letters, change all letters to lowercase.

Otherwise, change the first letter to uppercase and the remaining letters to lowercase.

Return the capitalized title.

Input Format

A single string containing multiple words containing both uppercase and lowercase characters separated by exactly one space character.

Each word in the string will contain at least one character.

Output Format

Return the capitalised string If the length of the word is 1 or 2 letters, change all letters to lowercase.

Otherwise, change the first letter to uppercase and the remaining letters to lowercase.

Constraints

The input string will only contain lowercase and uppercase alphabetic characters and spaces.

$1 \leq s.length \leq 10^4$

Each word in the string will have at least one character.

Adjacent words can be separated by one space character.

Input	Output
First leTTeR of EACH Word	First Letter of Each Word

Explanation:

The word "of" has length 2, so it is all lowercase.

The remaining words have a length of at least 3, so the first letter of each remaining word is uppercase, and the remaining letters are lowercase

Solution

```
#include <iostream>
```

```
#include <string>
```

```
#include<algorithm>
```

```
using namespace std;
```

```
string capitalizeTitle(string title) {
```

```
    string result = "";
```

```
    for (int i = 0; i < title.length(); i++) {
```

```
        string word = "";
```

```
        while (i < title.length() && title[i] != ' ') {
```

```
word += title[i];
```

```
i++;
```

```
}
```

```
if (word.length() <= 2) {
```

```
    transform(word.begin(), word.end(), word.begin(), ::tolower);
```

```
} else {
```

```
    transform(word.begin(), word.begin() + 1, word.begin(), ::toupper);
```

```
    transform(word.begin() + 1, word.end(), word.begin() + 1, ::tolower);
```

```
}
```

```
result += word + " ";
```

```
}
```

```
return result.substr(0, result.length() - 1);
```

```
}
```

```
int main() {
```

```
    string inputTitle;
```

```
    getline(cin, inputTitle);
```

```
    string outputTitle = capitalizeTitle(inputTitle);
```

```
    cout << outputTitle << endl;
```

```
return 0;  
}
```

Test Cases

<u>Input</u>	<u>Output</u>
capiTallze tHe titLe	Capitalize The Title
First leTTeR of EACH Word	First Letter of Each Word
i lOve leetcode	i Love Leetcode
a bc def ghijk	a bc Def Ghijk
tHe quIck bROWN foX	The Quick Brown Fox
capiTallze tHe titLe	Capitalize The Title

Ques 4 -

Given two binary strings a and b, return their sum as a binary string..

Input Format

First line : String a

Second line : String b

Output Format

Return a string containing their sum as a binary.

Constraints

1; 1 <= a.length, b.length <= 104

2; a and b consist only of '0' or '1' characters.

3; Each string does not contain leading zeros except for the zero itself.

Input	Output
11 1	100

Solution

```
#include <iostream>
```

```
#include <string>
```

```
#include<algorithm>
```

```
using namespace std;
```

```
string addBinary(string a, string b) {
```

```
    string result = "";
```

```
    int i = a.length() - 1;
```

```
    int j = b.length() - 1;
```

```
    int carry = 0;
```

```
    while (i >= 0 || j >= 0) {
```

```
        int sum = carry;
```

```
        if (i >= 0) {
```

```
            sum += a[i] - '0';
```

```
            i--;
```

```
        }
```

```
        if (j >= 0) {
```

```
            sum += b[j] - '0';
```

```
            j--;
```

```
        }
```

```
        carry = sum > 1 ? 1 : 0;
```

```
        result += to_string(sum % 2);
```

```
    }
```

```

    if (carry != 0) {
        result += to_string(carry);
    }

    reverse(result.begin(), result.end());

    return result;

}

int main() {
    string a, b;
    cin >> a >> b;

    string res = addBinary(a, b);

    cout << res << endl;

    return 0;
}

```

Test Cases

<u>Input</u>	<u>Output</u>

11 1	100
1010 1011	10101
1101 1011	11000
1111 1111	11110
10101 1101	100010

Section - 4 (2*10) Coding Questions 10 Marks

Q 1. You are tasked with designing a C++ program involving hybrid and hierarchical inheritance. Create a set of classes to represent different types of vehicles. Your program should include:

(i) A base class Vehicle with attributes make and year and a method displayDetails() that displays the make and year of the vehicle.

(ii) Two derived classes, Car and Bike, both inheriting from the Vehicle class. The Car class should have an additional attribute, fuelType, and the Bike class should have an additional attribute, engineType. Override the displayDetails() method in both derived classes to display their respective attributes as well.

(iii) Another class, Truck, which inherits from Car. This represents hierarchical inheritance. The Truck class should have an additional attribute, cargoCapacity, and override the displayDetails() method to display all attributes, including those inherited from the Car class.

(iv) Finally, create an instance of each of the Car, Bike, and Truck classes, and call their displayDetails() methods to display the details of these vehicles.

Write the C++ code for the above scenario.

Input Format

First line should contain 3 space separated values representing details of Car : make(string) , year (int) and fuel type(string).

Second line should contain 3 space separated values representing details of Bike: make(string), year(int), engine type(string).

Third line should contain 4 space separated values representing details of the truck: make(string), year(int), fuel type(string), cargo capacity(string).

Output Format

Details of Car, Bike and Truck on separate lines as shown in sample test case

Sample Test Case:

Input	Output
-------	--------

Toyota 2022 Petrol

Honda 2021 Electric

Ford 2020 Diesel 5

Car Details:

Make: Toyota

Year: 2022

Fuel Type: Petrol

Bike Details:

Make: Honda

Year: 2021

Engine Type: Electric

Truck Details:

Make: Ford

Year: 2020

Fuel Type: Diesel

Cargo Capacity: 5 tons

Solution:

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
class Vehicle {
```

```
protected:
```

```
    string make;
```

```
    int year;
```

```
public:
```

```
    Vehicle(const string& make, int year) : make(make), year(year) {}
```

```
    void displayDetails() {
```

```

        cout << "Make: " << make << endl << "Year: " << year << endl;
    }
};

class Car : public Vehicle {
protected:
    string fuelType;

public:
    Car(const string& make, int year, const string& fuelType) : Vehicle(make,
year), fuelType(fuelType) {}

    void displayDetails() {
        Vehicle::displayDetails();

        cout << "Fuel Type: " << fuelType << endl;
    }
};

class Bike : public Vehicle {
protected:
    std::string engineType;

public:
    Bike(const string& make, int year, const string& engineType) :
Vehicle(make, year), engineType(engineType) {}

```

```

void displayDetails() {
    Vehicle::displayDetails();

    cout << "Engine Type: " << engineType << endl;
}
};

```

```

class Truck : public Car {
protected:
    int cargoCapacity;

public:
    Truck(const string& make, int year, const string& fuelType, int
cargoCapacity)
        : Car(make, year, fuelType), cargoCapacity(cargoCapacity) {}

```

```

    void displayDetails() {
        Car::displayDetails();

        cout << "Cargo Capacity: " << cargoCapacity << " tons" << endl;
    }
};

```

```

int main() {
    string car_build,car_fuel;

```

```
int car_year;
```

```
cin >> car_build >> car_year >> car_fuel;
```

```
Car car(car_build, car_year, car_fuel);
```

```
string bike_build, bike_engine;
```

```
int bike_year;
```

```
cin >> bike_build >> bike_year >> bike_engine;
```

```
Bike bike(bike_build, bike_year, bike_engine);
```

```
string truck_make, truck_fuel_type;
```

```
int truck_year, truck_cargo_capacity;
```

```
cin >> truck_make >> truck_year >> truck_fuel_type >>  
truck_cargo_capacity;
```

```
Truck truck(truck_make, truck_year, truck_fuel_type,  
truck_cargo_capacity);
```

```
cout << "Car Details:" << endl;
```

```
car.displayDetails();
```

```
cout << "\nBike Details:" << endl;
```

```
bike.displayDetails();
```

```
cout << "\nTruck Details:" << endl;
```

```
truck.displayDetails();
```

```
return 0;
```

```
}
```

Test Cases

Input	Output
Toyota 2022 Petrol Honda 2021 Electric Ford 2020 Diesel 5	Car Details: Make: Toyota Year: 2022 Fuel Type: Petrol Bike Details: Make: Honda Year: 2021 Engine Type: Electric Truck Details: Make: Ford Year: 2020 Fuel Type: Diesel

Cargo Capacity: 5 tons

BMW 2019 Diesel

Enfield 2021 Petrol

Ford 2020 Diesel 5

Car Details:

Make: BMW

Year: 2019

Fuel Type: Diesel

Bike Details:

Make: Enfield

Year: 2021

Engine Type: Petrol

Truck Details:

Make: Ford

Year: 2020

Fuel Type: Diesel

Cargo Capacity: 5 tons

Chevrolet 2023 Diesel

BMW 2021 Petrol

Toyota 2020 Gasoline 5

Car Details:

Make: Chevrolet

Year: 2023

Fuel Type: Diesel

Bike Details:

Make: BMW

Year: 2021

Engine Type: Petrol

Truck Details:

Make: Toyota

Year: 2020

Fuel Type: Gasoline

Cargo Capacity: 5 tons

Ford 2023 Diesel
Suzuki 2020 Petrol
Toyota 1999 Gasoline 10

Car Details:
Make: Ford
Year: 2023
Fuel Type: Diesel

Bike Details:
Make: Suzuki
Year: 2020
Engine Type: Petrol

Truck Details:
Make: Toyota
Year: 1999
Fuel Type: Gasoline
Cargo Capacity: 10 tons

Nissan 2017 Diesel	Car Details:
Harley-Davidson 2020 Gasoline	Make: Nissan
Toyota 2015 Petrol 8	Year: 2017
	Fuel Type: Diesel
	 Bike Details:
	Make: Harley-Davidson
	Year: 2020
	Engine Type: Gasoline
	 Truck Details:
	Make: Toyota
	Year: 2015
	Fuel Type: Petrol
	Cargo Capacity: 8 tons

Q 2. Print elements according to pattern similar to number 5 :

Given a square matrix `mat[][]`, and you have to traverse the input matrix and print elements according to a specific pattern which is similar to integer 5.

Note -> you can take reference to sample input and output for better understanding which type of pattern you have to print using array elements.

Input Format

First line - Take size of array N.

Second line - Take N*N integer value to store them in a 2D array or square matrix.

Output Format

Print all the elements in specific order which came under a pattern similar to integer 5.

Output elements should print in a single line and space separated.

Constraints

$$3 \leq N \leq 11$$

$$0 \leq \text{arr}[i] \leq 10^3$$

Input	Output
3 7 1 5 2 9 4 11 8 0	5 1 7 2 9 4 0 8 11
5 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15	5 4 3 2 1 6 11 12 13 14 15 20 25 24 23 22 21

16 17 18 19 20	
21 22 23 24 25	

Explanation - in the first example we have to print all elements present on the pattern of 5. And it will print the first row right to left , then the first row (top+1) to (mid -1) index, then middle row in opposite direction , and so on.

In the second example when we print all elements present on the pattern of 2 are 5 4 3 2 1 then 6 then 11 12 13 14 15 then 20 then 25 24 23 22 21.

Solution

```
#include <iostream>
```

```
#include <vector>
```

```
#include <algorithm>
```

```
using namespace std;
```

```
void print(vector<vector<int> >& mat) {
```

```
    int n = mat.size();
```

```
    vector<int>ans;
```

```
    int count =0;
```

```
    for(int i =n-1; i>=0; i--){
```

```
        ans.push_back(mat[0][i]);
```

```
        count++;
```

```
    }
```

```
    for(int i =1; i<(n/2); i++){
```

```
ans.push_back(mat[i][0]);
```

```
count++;
```

```
}
```

```
for(int i = 0; i < n; i++){
```

```
ans.push_back(mat[n/2][i]);
```

```
count++;
```

```
}
```

```
for(int i = (n/2)+1; i < n-1; i++){
```

```
ans.push_back(mat[i][n-1]);
```

```
count++;
```

```
}
```

```
for(int i = n-1; i >= 0; i--){
```

```
ans.push_back(mat[n-1][i]);
```

```
count++;
```

```
}
```

```
for(int i = 0; i < count; i++){
```

```
cout << ans[i] << " ";
```

```
}
```

```
cout << endl;
```

```

}

int main(){

    int N;

    cin>>N;

    vector<vector<int> >ans(N, vector<int>(N));

    for(int i = 0; i < N; i++) {

        for(int j = 0; j < N; j++) {

            cin>>ans[i][j];

        }

    }

    print(ans);

    return 0;

}

```

Test Cases -

<u>Input</u>	<u>Output</u>
3 17 11 9 3 3 3 3 2 1	9 11 17 3 3 3 1 2 3

5 1 2 3 4 5 6 7 8 9 1 9 12 1 14 15 16 1 18 19 20 2 22 2 24 25	5 4 3 2 1 6 9 12 1 14 15 20 25 24 2 22 2
3 1 2 3 4 5 6 7 8 9	3 2 1 4 5 6 9 8 7
5 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25	5 4 3 2 1 6 11 12 13 14 15 20 25 24 23 22 21
3 9 8 7 6 5 4 3 2 1	7 8 9 6 5 4 1 2 3

3 1 2 3 4 1 2 3 4 1	3 2 1 4 1 2 1 4 3
3 9 9 9 9 9 9 9 9 9	9 9 9 9 9 9 9 9
3 12 10 2 3 3 4 2 8 3	2 10 12 3 3 4 3 8 2
3 1 2 4 3 6 9 5 10 15	4 2 1 3 6 9 15 10 5
5 1 2 3 4 5 1 2 3 4 5	5 4 3 2 1 1 1 2 3 4 5 5 5 4 3 2 1

1 2 3 4 5	
1 2 3 4 5	
1 2 3 4 5	